AOSP 进阶丛书 · 第十五册 · ANDROID 13 (API 33)

构建系统 Soong 实战

Soong 四段管线（Android.bp→ninja→编译→镜像）、device 树三层继承与 product 配置、构建变体（eng/userdebug/user）、Soong 模块类型速查、out/ 与增量优化实战。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：系统的"工厂" 2
第一章 构建管线：bp→ninja→镜像 2
第二章 device 树与 product 配置 3
第三章 Soong 模块类型速查 4
第四章 out/ 与增量优化 5
第五章 设计决策六问 5
附录 A 实战场景 5
附录 B 快问快答（12 条） 6
附录 C 自检题 6

本册速览

Soong 构建管线：Android.bp→ninja→image
device tree 继承链与 product 配置（lunch 体系）
构建变体 eng/userdebug/user 差异
Soong 模块类型速查（cc_binary/java_library/aidl_interface/apex…）
out/ 产物结构与增量构建实践

前言：系统的"工厂"

源码是原料，构建系统是**工厂**——本册教你读懂这座工厂：Soong 怎么读 Android.bp、lunch 怎么选产品、out/ 里产出了什么、增量怎么加速。读完你能回答"改一行 framework 怎么最快上机""为什么 lunch 选错了全编失败""out/ 占了 200GB 怎么清"这类实际问题。

第一章 构建管线：bp→ninja→镜像



图 1 Soong 四段：Android.bp→依赖图（ninja）→编译（clang/javac）→镜像打包（Makefile），常用命令族与 out/ 结构

1.1 四段逐段

text 构建管线（锚点+讲解注释）

```
① 描述层：Android.bp（声明式）+ Android.mk（Kati 遗留）
// build/soong/android/（Go 框架）解析 bp→ModuleContext
// 变体轴：arch(arm64/x86_64) × os(android/vendor) × link(shared/static) × APEX
② 依赖图：Soong→build.ninja（主目标文件）
// ninja 是"最小构建系统"——只管执行不管解析
// 一个 cc_binary 可能展开 90+ 目标（变体组合爆炸）
③ 编译：ninja -j N 调 clang/javac/dex2oat
// 增量：out/soong/.intermediates/ 按模块名+输入哈希缓存
// m services 只重编受影响模块链（30s-5min vs 全编 2-4h）
④ 镜像：build/make/core/Makefile（Kati 侧）
// system.img(ext4/erofs) · boot.img(mkbootimg) · *.apex · vbmeta(AVB)
// device/<oem>/<product>/BoardConfig.mk 决定尺寸/签名/分区
```

1.2 实用命令族

命令	用途	耗时参考
source build/envsetup.sh	加载函数	秒
lunch aosp_x86_64-userdebug	选产品	秒（首次需交互）
m	全编	2-4h（首次）
m services	只编 services.jar	30s-2min
m snod	只重打 system.img	~1min
mm	当前目录	秒-分钟
adb root; adb sync	推增量	秒
m nothing	验证语法	秒（不产出）
m module-info	依赖图 dump	分钟

1.3 Soong 框架逐段注释（Go 侧怎么展开变体）

text build/soong/android/module.go（结构+讲解注释）

```
// 讲解注释①：每个 Android.bp 模块类型对应一个 Go 结构体：
// cc_binary → AndroidModulesEntry{name:"cc_binary", ...}
// Soong 解析 bp 文件时反射调用工厂函数创建模块对象
// 讲解注释②：模块的 Variants() 方法返回所有需要展开的变体轴：
// [arm64-android, arm-android, x86_64-android, ...]
// × [vendor, ] × [shared, static] × [apex1, apex2, ...]
// → 每个"变体组合"生成一个独立的编译目标
// 讲解注释③：依赖解析（deps.go）：
// shared_libs: ["libbinder"] → 找到 libbinder 模块 →
// 按当前变体匹配 libbinder 的对应变体 → 生成 DT_NEEDED
// → ninja 文件记录依赖关系（改了 libbinder → 重链所有下游）
// 讲解注释④：产物（outputs）：
// 编译产物路径规则：out/soong/.intermediates/<path>/<variant>/<output>
// 每个模块独立目录——增量按输入哈希判断是否需要重编
```

设计洞察 · 逐段精读：为什么 Go 而不是 Python 或 C++

编译速度（Go 编译到内存→秒级解析数万 bp）+ 类型安全（struct/interface 模型匹配"声明式 bp→强类型模块"）+ 工具链统一（Go test/Go doc）。这是"工程语言选型服务于工程目标"的教科书案例。

第二章 device 树与 product 配置

`device/<oem>/<product>/` 是产品的"身份证"。三层继承：**chipset → device → product**（每层 device.mk 叠加 PRODUCT_PACKAGES 等变量）。lunch 的目标名由这里的 `AndroidProducts.mk` 声明。

text device 树结构（以 device/google/cuttlefish 为参考）

```
AndroidProducts.mk // 产品列表（lunch 可见的目标名）
device.mk // 本设备的包+配置+overlay
BoardConfig.mk // 板级：分区尺寸/签名/内核路径
sepolicy/ // 设备特有 SELinux 策略
rro/ // Runtime Resource Overlay（书1 §六.2）
```

2.1 构建变体

变体	用途	关键差异
eng	开发	root 可用/调试开/SELinux permissive
userdebug	调试发布	adb root 可用/SELinux enforcing
user	正式	无 root/全限制（量产唯一合法）

面试高频："userdebug 与 user 的差异不止 root——还有 ro.debuggable（影响 logcat/属性写）与 SELinux 模式"。

第三章 Soong 模块类型速查

模块类型	产出	典型用例
cc_binary	可执行	init/servicemanger/你的 HAL 服务
cc_library_shared	.so	libbinder/libutils
java_library	.jar	framework.jar/services.jar
android_app	APK	SystemUI/Launcher3
aidl_interface	四后端绑定	HAL 接口/系统服务接口
apex	APEX 模块	com.android.art/tethering
android_test	测试 APK	CTS 用例
genrule	生成文件	编译期代码/资源生成
filegroup	文件集	跨模块引用源码集
sdk_snapshot	SDK 库	Mainline 模块对 SDK 的固定引用

设计洞察 • 选型口诀

C/C++ 用 cc_*；Java 用 java_library 或 android_app；接口用 aidl_interface（自动四后端）；可独立升级用 apex；其他类型查 [Soong Modules Reference](https://source.android.com/docs/setup/build)。

第四章 out/ 与增量优化

text out/ 结构速查

```
out/target/product/<device>/ → 镜像（system.img/boot.img/…）
out/soong/.intermediates/<path>/ → 模块中间产物（.o/.jar/.dex）
out/host/linux-x86/bin/ → host 工具（adb/fastboot/soong_ui）
out/visible_modules/ → 模块可见性声明缓存
```

增量原理：Soong 为每个模块计算输入哈希；`m services` 只重编哈希变化的模块及下游依赖——编辑 framework/java → services.jar → system.img → adb sync，最快路径。**空间回收**：`m clean`（删 out/ 下构建产物，保留 .zip）；`rm -rf out`（彻底重来，首次全编 2-4h）。

第五章 设计决策六问

- **为什么声明式 bp 取代命令式 mk？** 变体由框架统一展开（mk 要手写 ifeq 分支）；声明式可静态分析校验（host_configverification）。
- **为什么 Kati 保留？** 数百万行 Make 规则不可能一夜迁完——androidmk 工具可自动转换，两者长期共存。
- **为什么 out/ 那么大？** 每个变体一套中间产物（90+ 目标×数万模块）；`m nothing` 验证后可 `rm out/soong/.intermediates` 只留缓存骨架。
- **为什么 Bazel 还在试验？** Soong 是 Google 内部产物闭源考量+Bazel 的 remote execution 更优——未来可能三引擎共存（bp+kati+bazel）。
- **为什么 device 树有 BoardConfig 而不用 bp？** 板级配置（分区尺寸/签名密钥）是"硬件事实"而非"构建逻辑"——mk 的简单变量足够且更稳定。
- **为什么 lunch 是交互式的？** 产品目标可能几十个（chipset×device×variant 组合），交互式选择比命令行全名更友好——`lunch` 不带参数列出全部。

附录 A 实战场景

场景	命令	耗时	
改 framework/base 一行	m services && adb sync	<2min	
加新系统 App	m MyApp && adb sync	<1min	
改 SELinux 策略	m system/sepolicy && adb sync	<1min	
新 HAL 五步验证	m android.hardware.mysvc && flash vendor	首次+5min	
验证 bp 语法	m nothing	秒	

场景	命令	耗时	
查模块依赖	m module-info	grep mymod	分钟
清理重编	m clean && m	2-4h	

附录 B 快问快答（12 条）

- Soong 的输入输出？——输入 Android.bp→输出 build.ninja。
- 一个 cc_binary 可能展开多少目标？——90+（arch×os×link×APEX 变体爆炸）。
- 增量靠什么？——out/soong 的模块输入哈希缓存。
- 最快验证语法？——m nothing。
- lunch 什么格式？——product-variant（如 aosp_x86_64-userdebug）。
- BoardConfig 管什么？——分区尺寸/签名密钥/内核路径。
- system.img 谁打的？——build/make/core/Makefile（Kati 侧）。
- APEX 构建用什么模块类型？——apex{}。
- AIDL 接口定义用什么？——aidl_interface{}（自动四后端）。
- userdebug 与 user 差异？——root+ro.debuggable+SELinux 模式。
- out/ 可删吗？——rm -rf out 完全重建（首次 2-4h）。
- device 树的继承链？——chipset→device→product（三层叠加）。

附录 C 自检题

- 画 Soong 四段管线，标出 ninja/Makefile 各管哪段。
- 改一行 ActivityManagerService.java 到手机上生效的最短路径。
- 为什么 m services 比 m 快？增量机制的本质。
- BoardConfig.mk 与 Android.bp 各自的职责边界。